DOCUMENTATION · PERFORMANCE

Performance, v2: an OLTP benchmark on PostgreSQL

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_V2. This copy was made on 4 November 2026.

> **Superseded by Performance, v3.** v3 reruns this page's 3-round blocks with 5 rounds, calibrates the round-trip count, adds sizing, and fixes the overstatements a review of this page found. The numbers below aren't wrong, but v3's are the ones to use.

The short answer. With one gunicorn worker (as in the Docker image) in front of PostgreSQL, a saved query by primary key at 32 concurrent callers:

	V0.41.0	`MAIN` (UNRELEASED 0.42)	`MAIN` WITH THE METADATA CACHE
Requests a second	682	743	875
One caller at a time, p50	1.43 ms	1.38 ms	1.05 ms
With a governed key (masking and a table allow-list)	171	693	802

- **The governance features cost about 7% now.** A key with masking and a table allow-list is about 4× faster than in v0.41.0.
- **One process serves 740–875 requests a second.** Latency holds until the load reaches about 75% of that.
- **More processes add throughput.** Eight workers reached 2,200–2,700 requests a second on four physical cores.
- **PostgreSQL is never the limit here.** At most it used 2 of the 4 hardware threads it had.
- **The remaining cost of a call** is about 1 ms of Python (logging alone takes a fifth of it), plus 4 round trips to the data database for every query. Over a real network, those round trips dominate.

Every number below is the median of 3–5 interleaved rounds with its min–max range. A difference counts as real only when the two ranges don't overlap. This page replaces the PostgreSQL parts of PERFORMANCE.md, whose method two reviews found too weak for its comparisons. What changed from v1 lists what was fixed, and what v1 got wrong.

Contents

- Method
- Results
- Where the time goes
- What to do with this
- What changed from v1
- Limits
- Run it yourself

Method

The machine, and how it was kept steady

One laptop: an Intel i7-12700H with 6 performance cores (CPUs 0–11, two threads each) and 8 efficiency cores (CPUs 12–19), 32 GB of memory, Ubuntu with kernel 6.8. In v1, the same code measured 2× apart on different runs. Most of that was the laptop: on battery, on a power-saving profile, and with nothing pinned, so the scheduler could move any part onto a slow efficiency core at any moment. This run:

- **On AC power, power profile `performance`,** CPU energy preference `performance`. Turbo stayed on, since turning it off needs root.
- **Each part on its own cores,** so they don't take CPU from each other:

PART	CPUS	HOW
QueryAPIGate (gunicorn)	0–7: four performance cores, both threads of each	`taskset -c 0-7`
PostgreSQL	8–11: two performance cores	`docker run --cpuset-cpus 8-11`
Load generator (oha) and pgbench	12–19: the efficiency cores	`docker run --cpuset-cpus 12-19`

- **No proxy in the path.** The gateway reached PostgreSQL at the container's own address, not through Docker's published port, whose userland proxy is another unpinned process copying every byte.

- **A floor before and after every block:** 10 s of `/health` on one worker, the same work every time, so a machine that changed speed shows up. Across the 2 h 25 min of the run it stayed between 1,934 and 2,296 requests a second, -13% to +3% of its median. The low end came in the last blocks (network, mixed, threads), which compare only within themselves. The CPU package ran at 65-100 °C, and the busiest gateway core's clock was sampled every 2 s of every run (2.4-4.4 GHz). Both are recorded per run.

Software: Python 3.12.3, gunicorn 26.2.0, Flask 3.1.1, psycopg2 2.9.10, sqlglot 25.34.1, PostgreSQL 16.15 (`postgres:16`), oha 1.16.0 pinned by digest (`ghcr.io/hatoo/oha@sha256:3ec3dbf549ea197793482d47a6324797411406bbf438c2fe8b91f244ec641a2f`).

The builds

LABEL	WHAT	COMMIT
v0.41.0	the 0.41.0 release	`f61d4ed`
main	`main` after the performance work in #88, unreleased - what 0.42 will ship	`c9b15c2`
main+cache	the same, with `QUERYAPIGATE_METADATA_CACHE_TTL=5`	`c9b15c2`

Each is exported from git (`git archive`) and run as the Docker image runs it: gunicorn with the `gthread` worker, 8 threads, the access log on, everything else at its default (so v0.41.0 has a pool of 5, `main` a pool of 8).

The database

pgbench's TPC-B-like schema at scale 100 (`pgbench -i -s 100`), 1.5 GB, fully in memory (`shared_buffers=2GB` , `max_connections=300` , `synchronous_commit=on`):

TABLE	ROWS	COLUMNS	KEY
`pgbench_accounts`	10,000,000	`aid integer` , `bid integer` , `abalance integer` , `filler character(84)`	primary key `aid`
`pgbench_branches`	100	`bid integer` , `bbalance integer` , `filler character(88)`	primary key `bid`
`pgbench_tellers`	1,000	`tid integer` , `bid integer` , `tbalance integer` , `filler character(84)`	primary key `tid`
`pgbench_history`	0	`tid` , `bid` , `aid` , `delta integer` , `mtime timestamp` , `filler character(22)`	none

QueryAPIGate's own store - keys, saved queries, connections, run history - is a second database, `qagstore` , in the same PostgreSQL, as `QUERYAPIGATE_DATABASE_URL` .

The calls and the keys

Three saved queries on one connection, `oltp` , called with a random id on every request:

SAVED QUERY	SQL	CALLED AS
`account` - a **lookup**	`SELECT aid, bid, abalance FROM pgbench_accounts WHERE aid = :aid`	`/q/account?aid=` 1 to 9,999,999
`account_page` - a **100-row page**	`SELECT a.aid, a.abalance, b.bid, b.bbalance FROM pgbench_accounts a JOIN pgbench_branches b ON b.bid = a.bid WHERE a.aid BETWEEN :start AND :start + 99 ORDER BY a.aid`	`/q/account_page?start=` 1 to 999,999 `&page_size=100`
`deposit` - a **write**	`UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid`	`/q/deposit?aid=` 1 to 9,999,999 `&delta=` 1 to 9

KEY	GRANTS
plain	`{"queries": ["account", "account_page"], "connections": []}`
governed	the same, plus `"allowed_tables": ["pgbench_accounts", "pgbench_branches"]` and `"mask": [{"column": "*.abalance", "action": "hash"}]`
writer	`{"queries": ["deposit"], "connections": [], "allow_writes": true, "allowed_write_ops": ["update"]}` , on a server with `QUERYAPIGATE_ALLOW_WRITES=1`

`benchmarks/oltp.py` creates all of them through the Management API. Every one of the run's 5.8 million requests returned 200.

The load

- **Closed loop**, for throughput: `oha -c N` keeps N callers, each sending its next request when the last one answers. Each run is 30 s, after a 5 s warm-up at 8 callers that also fills the pool and the governance caches.

- **Open loop**, for latency against load: `oha -q R --latency-correction` sends R requests a second whatever happens, and measures each from when it should have been sent. That avoids coordinated omission, where a closed loop under-reports latency at saturation because a slow answer delays the next request. Rates of 25%, 50%, 75%, 90%, 100% and 110% of the build's closed-loop capacity, 20 s each, 128 connections available.
- **Interleaved.** Within a block, every round runs every build (or setting) once, in an order rotated each round - A B C, then B C A, then C A B - so none always runs first, warmest or coolest.

What is recorded for each run

- Requests a second, the success rate and every status code; p50, p90, p99 and p99.9 latency (oha's own percentiles).
- **CPU, in cores:** the gateway's from `/proc` (master and workers), PostgreSQL's from its container's cgroup (`cpu.stat`), each divided by the run's wall time.
- **Transactions per request** on the store and on the data database, from `pg_stat_database`. These are exact for 30 s runs; for exact *statement* counts, `log_statement = 'all'` was turned on for a few hundred requests (see Round trips).
- The busiest gateway core's clock and the CPU package temperature.

All of it is in `benchmarks/results/oltp-postgres-2026-10-23/`: `results.jsonl` (one line per run), `results.md` (every table, generated by `benchmarks/oltp_report.py`), the per-second latency of the cold-path runs, and the py-spy profiles.

Results

The database on its own

pgbench from the load generator's cores, 5 rounds of 10 s:

WORKLOAD	CLIENTS	TRANSACTIONS A SECOND	AVERAGE LATENCY
Primary-key read (`pgbench -S`)	1	38,289 (37,465-39,586)	0.026 ms
Primary-key read (`pgbench -S`)	32	112,321 (96,140-123,670)	0.285 ms
TPC-B: three updates, a select, an insert	32	12,970 (12,617-15,293)	2.5 ms

A lookup costs PostgreSQL 0.03 ms on two cores. Everything a call through QueryAPIGate takes beyond that is the gateway and its round trips.

One process, 32 callers

5 rounds of 30 s per cell. **Cached** is `main` with the metadata cache on.

CALL		V0.41.0	MAIN	CACHED
Lookup, plain key	req/s	682 (679-686)	743 (736-746)	875 (869-876)
	p99	55.8 ms	54.1 ms	45.1 ms
Lookup, governed key	req/s	171 (171-172)	693 (691-694)	802 (800-806)
	p99	223 ms	57.1 ms	49.9 ms
100-row page	req/s	596 (595-598)	639 (638-642)	738 (737-740)
	p99	64.8 ms	61.4 ms	52.7 ms
Write	req/s	682 (679-683)	738 (733-743)	875 (872-881)
	p99	56.9 ms	53.9 ms	45.9 ms

All the ranges are clear of each other:

- **main against v0.41.0:** +7 to 9% for plain keys, from reading the store less (below); +304% for a governed key, from the masking and table-check caches.
- **Cached against main:** +16 to 19%, on every call.
- **The gateway used 1.2 cores for one worker:** one for Python, the rest for the threads' I/O. PostgreSQL used 0.2-0.4.

One caller at a time

The latency of a single call, 5 rounds of 30 s:

CALL		V0.41.0	MAIN	CACHED
Lookup, plain key	p50	1.43 ms (1.43-1.44)	1.38 ms (1.35-1.42)	1.05 ms (1.04-1.06)
	p99 / p99.9	2.1 / 3.0 ms	2.2 / 3.1 ms	1.7 / 2.4 ms
Lookup, governed key	p50	6.00 ms (5.79-6.06)	1.48 ms (1.46-1.51)	1.20 ms (1.16-1.20)
	p99 / p99.9	7.6 / 13.6 ms	2.4 / 3.3 ms	1.9 / 2.5 ms

For one plain caller, main and v0.41.0 overlap: three fewer store reads don't show at this speed. The cache's -24% does. A governed call went from 6 ms to 1.5.

Latency against offered load

The lookup, open loop, latency-corrected, 3 rounds of 20 s per rate. Capacity is each build's closed-loop rate above (743 and 875 requests a second):

OFFERED	MAIN: P50 / P99 / P99.9	CACHED: P50 / P99 / P99.9
25% of capacity	2.0 / 2.9 / 12.0 ms	1.5 / 2.3 / 7.8 ms
50%	1.6 / 2.7 / 6.1 ms	1.2 / 2.0 / 8.7 ms
75%	1.5 / 3.5 / 13.6 ms	1.2 / 3.4 / 14.0 ms
90%	1.6 / 7.9 / 15.6 ms	1.2 / 12.2 / 19.4 ms
100%	1.7 / 18.4 / 25.3 ms	1.7 / 15.0 / 23.3 ms
110%	411 / 1,379 / 1,396 ms - served 765 of 816	347 / 1,047 / 1,062 ms - served 917 of 962

- **p50 stays flat** right up to capacity. **p99 holds under 4 ms to 75%,** then climbs: 2× at 90%, 5× at 100%.
- **Past capacity, a queue builds.** Latency goes to seconds within the 20 s run, and only the capacity's worth of requests is served.
- **For capacity planning,** that means about 75% of the closed-loop number: about 550 requests a second per worker without the cache, 650 with it, for p99 under 4 ms.
- The lightest loads show a higher p99.9 than 50%: at a trickle, an idle core clocks down between requests.

Scale-out

Workers in one gunicorn (each a process, as each container would be), 64 callers, 3 rounds of 30 s:

WORKERS	LOOKUPS REQ/S: MAIN / CACHED	WRITES REQ/S: MAIN / CACHED	P99 LOOKUPS: MAIN / CACHED	GATEWAY CORES	POSTGRESQL CORES: MAIN / CACHED
1	747 / 878	742 / 873	99 / 86 ms	1.2	0.29 / 0.17
2	1,214 / 1,472	1,202 / 1,448	69 / 62 ms	2.4	0.56 / 0.31
4	1,742 / 2,156	1,734 / 2,114	67 / 61 ms	4.6	1.04 / 0.59
8	2,241 / 2,741	2,200 / 2,617	148 / 105 ms	7.2	1.64 / 0.88

The cache's lead is clear at every size: +17% to +24%.

- **Throughput grows with processes, but not linearly here,** and the table shows why: requests a second per gateway core fall from 623 at 1 worker to 310 at 8. All 8 workers share CPUs 0-7, which are only four physical cores. From 4 workers on, each pair shares a core's two hyperthreads, and a hyperthread is worth far less than a core. Containers on their own cores, or their own hosts, would scale better than this. This run can't show by how much.
- **The cache halves PostgreSQL's work.** At 8 workers it used 1.64 cores without the cache and 0.88 with it - most of PostgreSQL's load was QueryAPIGate's own store, not the queries.
- **p99 at 8 workers** rises because 8 processes, 64 callers, and the gateway's own threads all contend for 8 hardware threads.

Threads per worker

One worker, the lookup, 32 callers, threads interleaved, 3 rounds:

THREADS	REQ/S	P99	GATEWAY CORES
1	778 (777-828)	52.5 ms	0.88
2	832 (831-902)	49.1 ms	1.12
4	763 (760-763)	53.3 ms	1.18
8	723 (722-725)	56.4 ms	1.20

A second thread helps a little: it runs Python while the first waits on the database. **Beyond two, throughput falls.** Eight threads, the Docker image's setting, are 13% slower than two on this workload, because they compete for Python's global interpreter lock - and, the profile shows, for the logging module's lock. Threads aren't for CPU. They are for calls that wait: slow queries, `?stream=true` exports, and `/events` connections that stay open. v1 said "they don't add CPU" and cited +42% from 1 to 2 threads. That figure came from the unstable laptop; this one doesn't.

The cold path: every 60 seconds

150 s of the lookup at 8 callers, each request's duration taken from gunicorn's access log (`%(D)s`):

KEY	REQUESTS	P50	P99	P99.9	P99.99	SLOWEST
governed	106,741	9.25 ms	14.66 ms	19.2 ms	73.7 ms	146.9 ms
plain	115,314	8.58 ms	13.57 ms	16.9 ms	20.9 ms	26.0 ms

- **The governed key's slow seconds come every 60 s:** t=1 (147 ms), t=61 (76 ms) and t=121 (74 ms). In a typical second, the slowest request takes 17 ms. Those are the moments QueryAPIGate re-reads the connection's schema and re-traces the statement for masking, which it does every 60 seconds.
- **Some callers pay for it:** the few that arrive while it happens wait about 60 ms more. It shows at p99.99, not at p99.9.
- **The plain key** has no table check or masking to redo, and no such spikes.

The access log

`main`, the lookup, 32 callers, gunicorn's access log on and off, interleaved, 5 rounds:

ACCESS LOG	REQ/S	P50	P99
on (the Docker image's default)	736 (735-739)	43.0 ms	54.4 ms
off	785 (782-792)	40.2 ms	51.7 ms

The access log costs 7%. QueryAPIGate also logs two lines of its own for each request, at INFO, which can't be turned off. The profile below puts all logging at a fifth of the gateway's CPU.

A database across a network

netem on the database container delays everything PostgreSQL sends by 0.5 ms, which is a same-zone cloud round trip (`tc qdisc add dev eth0 root netem delay 0.5ms`, ping 0.04 → 0.56 ms). The lookup, 3 rounds of 30 s:

	V0.41.0	MAIN	CACHED
One caller: req/s	132 (132-134)	161 (159-162)	270 (268-270)
One caller: p50	7.63 ms	6.24 ms	3.69 ms
32 callers: req/s	677 (672-752)	741 (735-750)	860 (857-870)

- **One caller pays for every round trip.** At 0.5 ms each, a call went from 1.4 to 6.2 ms on `main`. Fewer store reads made `main` 22% faster than v0.41.0. The cache made it 67% faster again, and halved single-call latency to 3.7 ms.
- **With 32 callers, round trips overlap,** and the CPU is the limit again: the same order as without a delay. v0.41.0's range overlaps `main`'s here, because one of its rounds ran fast.
- This is a simulation: one-way delay, no packet loss or jitter. Read it as how many round trips a call makes, not as a cloud measurement.

Mixed reads and writes, and hot rows

`main`, 3 rounds of 30 s:

WORKLOAD	REQ/S	P50	P99
Reads (24 callers) and writes (8 callers) at once	547 + 181 = 728	43.3 / 43.6 ms	54.8 / 54.9 ms
Writes on 10 million accounts (32 callers)	734 (729–734)	43.1 ms	55.0 ms
Writes on 99 hot accounts (32 callers)	735 (731–740)	43.0 ms	54.7 ms

- **Mixing reads and writes changes nothing:** they share the process's capacity in proportion.
- **Writes on 99 hot rows cost the same as on 10 million.** One worker sends at most 8 at once, so PostgreSQL's row locks never queue long enough to show. With many workers, or a slow transaction, they would.

Where the time goes

Round trips per call

Counted exactly: `log_statement = 'all'` on each database, then 100 calls after a warm-up.

STATEMENTS PER CALL	V0.41.0	MAIN	CACHED
Store, plain key or writer	7: 5 reads, 2 inserts	3: the key, the query, the connection	0.1: history, in batches
Store, governed key	8	4	1 - the masking hash key
Data database, any call	4: `BEGIN READ ONLY`, `SET LOCAL statement_timeout`, the query, `ROLLBACK`	4	4

- **History and events** are written in batches of up to 500 by a background thread, about once a second. That's about 0.01 statements per call in `main`, against one insert per row in v0.41.0.
- **The governed key reads the masking hash key on every call,** even with the cache on.
- **A query takes four round trips** to the data database: a transaction, its time limit, the statement, and the rollback (or commit, for a write). That's what the network test above measures.

The CPU

py-spy at 200 samples a second across the worker's threads, during 30 s of the lookup at 32 callers. Each sample is counted under the innermost frame that falls in one of these categories:

WHERE THE SAMPLES FALL	MAIN	CACHED
Store layer: queries, decoding rows, checks	28.2%	13.4%
Logging: the access log and QueryAPIGate's own lines	18.4%	20.4%
gunicorn, sockets, HTTP	16.8%	21.0%
Data query: the driver and the pool	12.3%	15.4%
Flask and Werkzeug	9.9%	11.9%
QueryAPIGate's request handling	4.6%	5.6%
History writer thread	3.0%	3.7%
Authentication, limits, quotas, metrics	2.5%	2.8%
SQL checks, parameters, masking	1.7%	1.9%
JSON	1.3%	2.6%
Other	1.3%	1.3%

The busiest single frames say more:

- **`logging` lock acquisition: 8–10%.** Eight threads write log lines through one handler, and wait on its lock - the same contention that makes more threads slower.
- **`store.refuse_store_access`: 7–10%.** A safety check that a connection doesn't point at QueryAPIGate's own SQLite file. It resolves file paths (`os.path.realpath`, several system calls) on every call - even for a PostgreSQL connection, which has no file to point at. With the cache on, it is the largest single cost in the store layer.

- **The store's `execute` : 17% in main, gone with the cache** - the three reads.
- **gunicorn's per-request worker heartbeat (`workertmp.notify`): 3%.**

What to do with this

For running QueryAPIGate:

- **Plan on about 550 requests a second per worker for fast OLTP queries,** or 650 with the metadata cache. That's 75% of capacity, where p99 stays under 4 ms. Then measure your own queries, which will usually cost more than these.
- **Turn on `QUERYAPIGATE_METADATA_CACHE_TTL`** if a few seconds' delay is acceptable for a change made on another instance to reach this one. In return, a worker serves 16-24% more, PostgreSQL does half the work, and across a network a single call takes 40% less time. Its trade-off is in PERFORMANCE.md.
- **For fast queries, try `--threads 2` to `4`.** Keep 8 if calls wait on slow queries, exports or `/events`.
- **Turn off gunicorn's access log** if a proxy in front already logs requests: +7%.
- **Scale with processes on their own cores.** Hyperthreads gave about half a core each here.

For QueryAPIGate itself - findings worth an issue each, in the order I'd take them:

1. **Fewer round trips per query.** Set `default_transaction_read_only` and `statement_timeout` once per pooled session, or send `BEGIN` and `SET LOCAL` in one round trip with the query. That's 4 round trips down to 1 or 2, and over a network, the largest remaining part of a call.
2. **Skip `refuse_store_access` 's path checks for server databases.** Only SQLite, DuckDB and H2 open a file. That would save 7-10% of CPU on every PostgreSQL, MySQL, ClickHouse and Trino call.
3. **Cache the masking hash key.** It's read on every governed call and almost never changes.
4. **Make logging cheaper:** a setting to skip the per-request INFO lines, or a queue handler so threads don't wait on one lock. That's up to a fifth of the CPU.
5. **Refresh the schema and masking trace in the background**, before they expire, so no caller waits 60-140 ms every minute.

What changed from v1

What the reviews raised, and what this run did about it:

RAISED	DONE
A laptop whose speed changes 2×	AC power, the `performance` profile, every part pinned to its own cores, a `/health` floor around every block (-13% to +3% over 2.5 hours), clock and temperature per run
Single runs compared against each other	3-5 interleaved, rotated rounds per cell; median and min-max; a difference counts only when the ranges don't overlap
Closed-loop load only, which hides latency at saturation	Open-loop runs at fixed rates, latency-corrected, from 25% to 110% of capacity
10 s runs that never cross the 60 s expiry, and no p99.9	30 s runs with p99.9; 150 s runs timed per request, which show the 60 s spikes
No network hop	A 0.5 ms simulated round trip
The threads claim contradicted its own data	Measured, interleaved: two threads are best here, eight are 13% slower
The 2-worker row looked like a regression	It was one noisy run. Interleaved, every row is clear
Throughput only in the scaling table	Latency, gateway and database CPU, and throughput per core
No profile of the time left	py-spy, broken down by category and by frame
Access log cost unknown	7%
Only the lookup was reproducible	All three queries and keys are spelled out above, and the harness creates them
" `main` after 0.41.0" isn't a version	Commits, versions and the oha image digest are recorded with the results

Two things v1 got wrong:

- **The store transactions per call.** v1 said about 5.5 before its changes and 2.6 after. Counted exactly, it's 7 statements before (5 reads and 2 inserts) and 3 after. PostgreSQL holds back an idle session's statistics for up to 10 seconds, and v1 read them about 2 seconds after its run ended.
- **Its PostgreSQL throughput figures were about half these.** That's the laptop's power state, not the code. The same build served 368 requests a second for v1 and 743 here.

Limits

- **One machine.** The gateway, the database and the load generator share it, on separate cores but one memory bus and one cooling system. Four physical cores for the gateway limit what scale-out can show.
- **No TLS, no load balancer or proxy, no Redis**, and so no shared rate limits. Each would add to the cost of a call.
- **The network is simulated**, one way, without jitter or loss.
- **Uniform random keys.** Real traffic is skewed. Hot rows were tested for writes, but not as a skewed read pattern, which would matter to a cache in front of the database.
- **Small results only:** one row, or a page of 100. Large results, `?stream=true`, the MCP server, and other databases weren't measured.
- **p99.99 and the slowest requests** come only from the 150 s runs. Elsewhere, 30 s runs give about 20,000 samples, enough for p99.9.
- **py-spy counts samples of threads it considers active,** and a thread waiting for the interpreter lock can count. The shares say where time is spent, not exactly how much each part would save.

Run it yourself

Needs Docker, `taskset`, and QueryAPIGate's server, PostgreSQL and masking dependencies in the Python that runs the script. py-spy is optional, for the profile:

```shell
pip install -e ".[server,postgres,flow]" py-spy
python benchmarks/oltp.py --out /tmp/qag-oltp setup all teardown     # about 2.5 hours
python benchmarks/oltp_report.py /tmp/qag-oltp/results.jsonl        # writes results.md beside it
```

- **Cores and refs:** `--gateway-cpus`, `--db-cpus` and `--load-cpus` take `taskset`-style lists; check your own machine's layout with `lscpu -e`. `--base` and `--head` choose the two git refs (v0.41.0 and `origin/main` by default).
- **Fewer blocks:** name them instead of `all`, e.g. `setup env b1 b2 teardown`. `--smoke` runs every block for 2 s to check the setup in about 10 minutes.
- **What it leaves behind:** `teardown` removes the database container and its data volume, about 2 GB.